

SWE30003 - Journal

Table of Contents

1.	Software Development Life Cycle and Requirements	6
1.1.	Verification v.s. Validation	6
1.2.	Types of requirements:	7
1.2.1.	Requirements	7
1.2.2.	Non-functional requirements	7
1.3.	Requirements Engineering	8
1.3.1.	Requirements Elicitation	9
1.3.2.	Requirements Analysis	9
2.	Software design	9
3.	Abstractions and Models	10
3.1.	Models	10
3.2.	Abstractions	11
3.3.	Models in abstractions	11
4.	Scenarios and use cases	11
4.1.	User perspective	11
4.2.	Use cases	12
4.3.	Actors	13
5.	Goal-Design Scale	13
6.	Domain Model Development	13
7.	Task Descriptions	14
8.	Workflows	14
9.	Object Oriented Design	15

9.1.	Objects and classes	15
9.2.	Object-Oriented Design Process	16
9.3.	Finding classes	17
9.3.1.	CRC Cards	17
9.4.	Responsibilities and Assigning Responsibilties	17
9.4.1.	What are responsibilities?	17
9.4.2.	Assigning Responsibilities	18
9.5.	Relationships between classes	18
9.5.1.	Example relationships	18
9.5.2.	Association and Aggregation (and Composition)	19
9.5.3.	Collections	19
9.6.	Inheritance and Abstract Classes	20
9.6.1.	Inheritance	20
9.6.2.	Abstract Classes	20
9.6.3.	How to design abstract classes	20
9.7.	Colaborations	21
9.7.1.	Roles and Protocols	21
9.8.	Design by Contract	22
9.8.1.	Abstract State v.s. Concrete State.	23
9.8.2.	Command-Query seperation principle.	23
9.9.	Decomposition	23
9.9.1.	Functional v.s. Object-oriented Decomposition	24
9.10.	Coupling and Cohesion	25
10.	Patterns	26
10.1.	What are patterns?	26

10.2. What do patterns solve?	26
10.3. Idioms, Frameworks, Libraries	27
10.4. Comparison of definitions	27
10.5. Design Patterns	28
10.5.1. Common Patterns	29
10.6. Other kinds of patterns	29
11. Software Architectures	30
11.1. Subsystems, Modules, Components	30
11.2. Why use architectures?	31
11.3. Architectural Styles	31
11.3.1. Layered Architectures	32
11.3.2. Dataflow Architecture	32
11.3.3. Repository Architectures	33
11.3.4. Client-server Architectures	34
11.3.5. Event-driven architectures	34
11.3.6. Peer-to-peer architectures	35
11.4. Architectural Tactics	35
11.5. Other	36
11.5.1. Heterogenous Styles	36
11.5.2. Reference Models and Architectures	36
12. Web Services	37
12.1. Definitions	37
12.2. Concepts	37
12.3. Service-Oriented Architecture (SOA)	39
12.4. Web application architectural styles and middleware	40

12.4.1. Web Service Standard: WS*, SOAP and WSDL	40
12.4.2. REST: (Representational State Transfer)	43
12.4.3. Microservices	44
13. Documentation	44
13.1. Purpose of Documentation	44
13.2. View:	45
13.2.1. Categories of Views	46
13.2.2. Kruchten's 4+1 View	47
13.3. IEEE Standard 1471	48
13.4. Notations	48
13.5. Design Evaluation	49
13.5.1. Architecture Tradeoff Analysis Method	50
13.5.2. Quality Utility Tree	51

List of figures

Figure 1 Software Development Life Cycle	6
Table 1 Requirement Types	7
Figure 2 Requirements Engineering process	8
Figure 3 Software Design Process	10
Figure 4 Sample UML diagram	16
Table 2 Example CRC Card	17
Table 3 Some kinds of relationships between classes	19
Table 4 Command-Query Separation Principle	23
Figure 5 Decomposition Process	24
Table 5 Functional v.s. Object-oriented Decomposition	24

Table 6	Coupling and Cohesion	25
Table 7	Some types of relevant coupling	25
Table 8	Definition of Idioms, Patterns, Libraries and Frameworks . . .	27
Figure 6	Visualization of Layered Architecture	32
Figure 7	Visualization of Dataflow Architecture	32
Figure 8	Visualization of Repository Architecture	33
Figure 9	Visualization of client-server architecture	34
Figure 10	Visualization of client-server architecture	34
Figure 11	Visualization of Repository Architecture	35
Figure 12	Sync vs Async communication	38
Figure 13	SOA Visualization	39
Figure 14	Message Exchange Patterns in WS*	42
Table 9	Categories of views	46
Figure 15	Visualization of Kruchten's 4 + 1 view	47
Figure 16	Visualization of ATAM	50

1. Software Development Life Cycle and Requirements

The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.

The SDLC is described below:

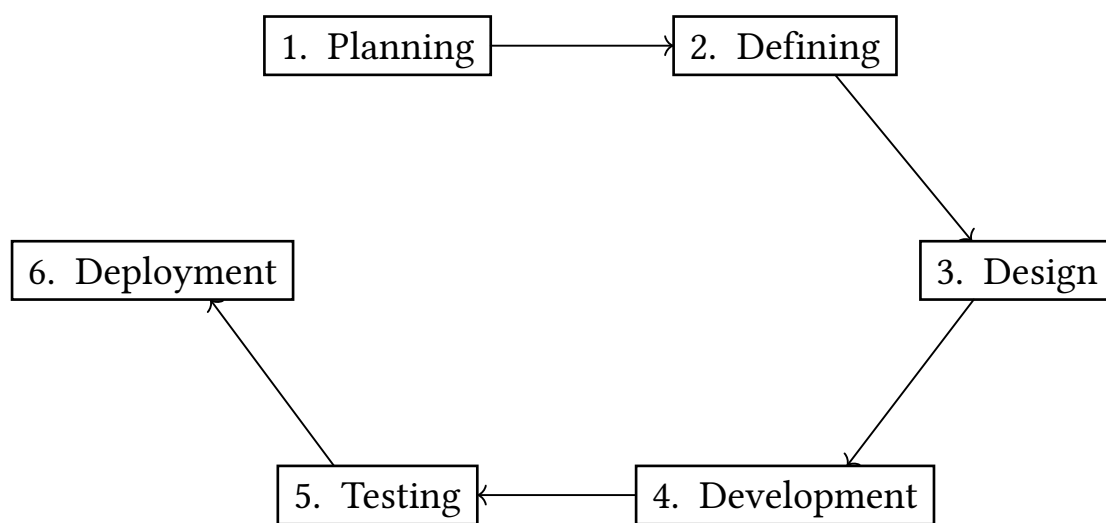


Figure 1: Software Development Life Cycle

In planning, the team must define what they are building and why (S1). Then, the specific product requirements must be defined and documented (S2).

1.1. Verification v.s. Validation

Verification is the process of **ensuring** that the software work products (such as requirements, design, and code) **conform to specified requirements and standards**. It focuses on **checking correctness and consistency** without necessarily executing the software. Verification asks: “Are we **building the product right?**”

Validation is the process of **evaluating the software** during or after development to **ensure it fulfills its intended use** and meets user and stakeholder needs. It typically involves executing the software and

observing its behavior in real or simulated environments. Validation asks: “Are we **building the right product?**”

1.2. Types of requirements:

Only functional v.s. non-functional

1.2.1. Requirements

- **Functional requirements** are requirements that describe **system services and functions**
- **Non-functional requirements** (a.k.a **quality requirements**) are **constraints** on the services and/or the development process
- **Domain requirements** stem from application domain of a system, which might be functional/non-functional

1.2.2. Non-functional requirements

There are various types of non-functional requirements:

Requirement Type	Definition	Example
Product Requirement	Specify how the system should behave or perform (e.g. speed, reliability, functionality)	latexmk must call the LaTeX compiler enough times to resolve all references
Organizational Requirement	Requirements derived from organizational policies, standards, or processes	The system must conform to the process defined in an internal policy document
External Requirement	Requirements arising from external factors such as laws, regulations, or interoperability needs	The system must allow users to check whether their personal data is stored, in compliance with data protection laws

Table 1: Requirement Types

1.3. Requirements Engineering

Requirements Engineering is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system.

The requirements engineering process is as following:

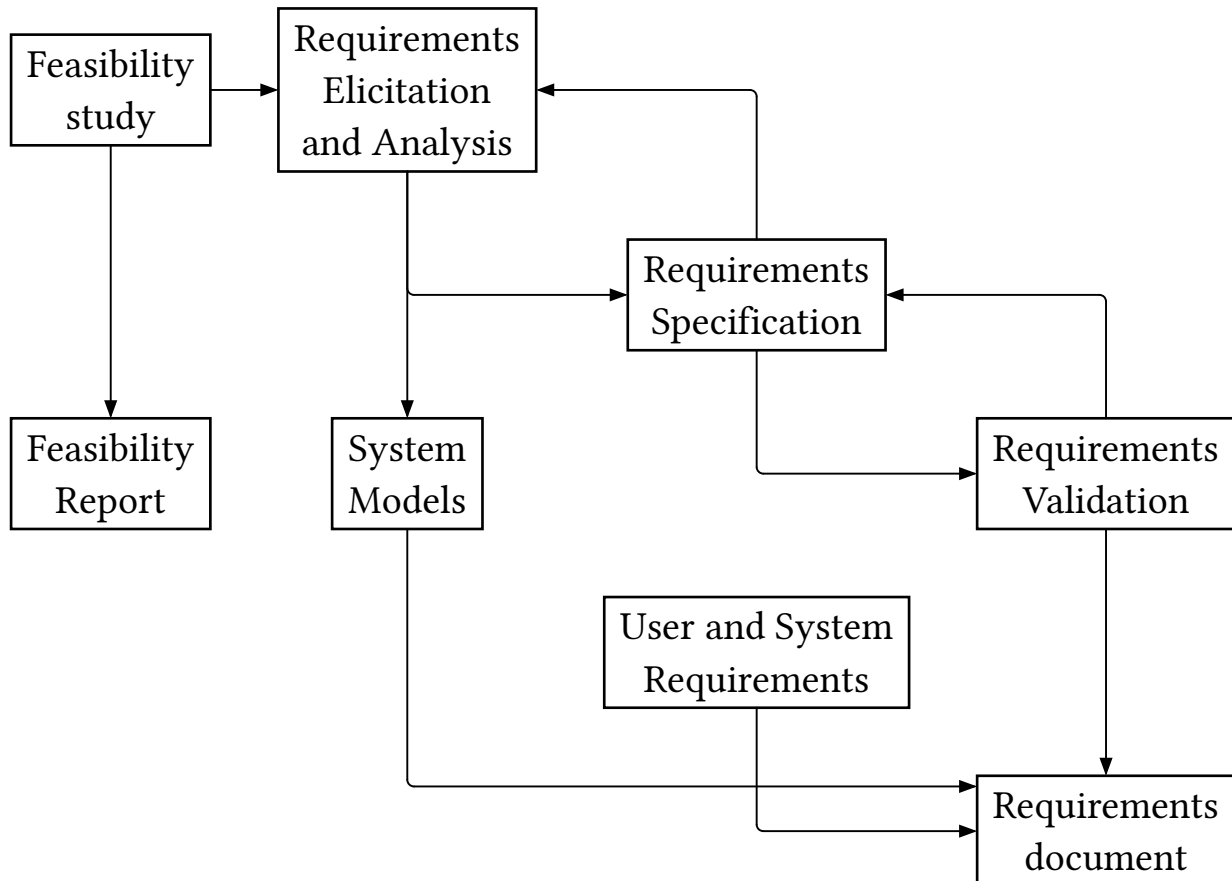


Figure 2: Requirements Engineering process

and has some activities:

Activity	Function
Feasibility study	Determine if the user needs can be satisfied with the available technology and budget.
Requirements elicitation	Find out what system stakeholders require from the system

Activity	Function
Requirements analysis	Clarify/define/document the requirements in a form understandable to the customer
Requirements specification	Define the requirements in detail. (Written as a contract between client and contractor.)

1.3.1. Requirements Elicitation

Sometimes called **requirements discovery**, it involves expert working with customers to define **domain**, **services** (functionality + quality), and operational **constraints**. It involves various **stakeholders**, such as end-users, managers, engineers, domain experts, etc.

1.3.2. Requirements Analysis

Requirements Analysis require **collaboration** of people with different backgrounds, such as **users** with domain knowledge, and **developers**, and it bridges the gaps between them.

2. Software design

The design of a system determines a set of components and inter-component interfaces that satisfy a specified set of requirements.

Software Design can be seen as **translating and refining** a (problem) domain model into abstractions that can be directly mapped to an implementation in a programming language.

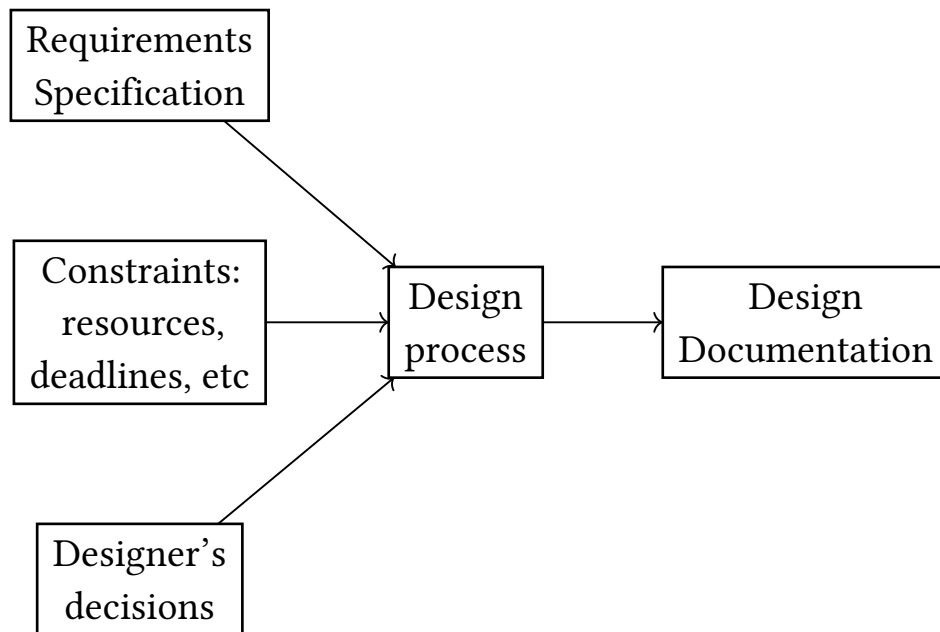


Figure 3: Software Design Process

The role of software design is to **bridge the gap** between **requirements** and **implementation**. It assists software engineers to develop software solution to a given set of requirements by help enabling **early verification** of software solutions, and to produce a **blueprint** for implementation. It also **enhances the communication** between project stakeholders.

Design Patterns conduct standard solutions to common software design problems.

3. Abstractions and Models

3.1. Models

Models are **simplification of reality**. Models are created to better understanding of the problem, the domain, the system to be built.

A model can capture and represent:

- Structure (static model)
- Behavior (dynamic model)

Example:

- Architectural blueprint
- Drawn UI
- Class diagrams

3.2. Abstractions

Abstractions are **concepts or ideas** not associated with any specific instance. It leaves out irrelevant detail, and highlights important events.

Some examples of abstractions:

- Instruction
- Condition
- Subsystems
- Packages
- Interfaces
- Abstract data types
- etc

3.3. Models in abstractions

Models are abstractions, in the sense that they are created for better understanding.

A domain model is a model that **captures the domain entities / concepts relevant to the original problem**. A **domain vocabulary** defines the meaning / interpretation of domain entities. The relationships between domain entities define an *abstraction over the problem domain*.

4. Scenarios and use cases

4.1. User perspective

Users tend to think about systems in terms of “features” (what system can do), e.g. the system must provide a function to *X*. They should tell

stories involving those features, and **scenarios** and **use cases** can assist to get requirements complete and consistent.

Scenarios: Users interact with a computer systems to complete a **task** or achieve a **goal**, which can be captures as a set of scenarios, i.e.

Mary browses the catalogue and adds desired items (chocolate) to the shopping basket. When she has finished shopping, Mary provides her address for delivery, credit card information and then confirms the transaction. The system confirms her payment and e-mails a transaction record.

— Example user story

We note that we covered the **ideal interaction** (everything works properly). But:

- What happens if the credit-card payment fails?
- What if the customer is a repeat customer, so shipping details are already in the system?

The key scenario remains, but to answer the questions we have to add *alternative* scenarios.

Scenarios can also be written down as a series of steps, and we need to identify scenarios that can be written up.

4.2. Use cases

If we structure scenarios, they can represent the behavioural requirements of the system, *from the users' viewpoint*. Every use case is directly or indirectly *invoked by an actor*, and must be *tied to a specific goal*.

4.3. Actors

An **actor** is a **role the user plays** when they interact with a system. They *perform* use cases. Actors can have multiple use cases, and single use case may have several actors.

Actors don't have to be human.

5. Goal-Design Scale

- *Goal Level* requirements satisfy business goals, e.g. calculations hit 5%
- *Domain Level* requirements satisfy the activities that go on outside the product, e.g. products support cost recording and quotation with experience data, e.g. product supports cost recording and quotation with experience data
- *Product Level* requirements specify what should come in and out of the product, e.g. product must have recording and retrieval function for experience data
- *Design level* requirements specify interface details between the system and other entities, interacting with it (such as users and other systems) and internals of the system, e.g. System shall have screen pictures

6. Domain Model Development

Also called *Domain Modelling*, the process is as follows:

1. Perform a **textual analysis** of the problem by understanding the problem domain; *circling all nouns* and *underlining all verbs*
2. Write down relevant **domain entities**, which is normally some of the nouns defined above
3. Revisit the specification to extract the **associations**
4. Record **associations** with the corresponding **domain entities**

7. Task Descriptions

Each Task Description has:

- **Purpose:** What the task aims to **achieve**
- **Trigger:** The **event that starts** the task
- **Precondition:** The condition that allows the task to occur
- **Frequency:** **How often** the task occurs
 - **Critical:** Special **high-pressure or large-scale** situations
- **Subtasks:** A list of steps that make up the task
 - Written in **neutral language**.
 - Describe what the user and computer must accomplish *together*.
- **Variants:** **Special cases or alternative ways** to perform subtasks.

Good tasks should have:

- All events covered
- Critical tasks covered

8. Workflows

A **workflow** is a coherent representation of the **(temporal)** dependencies between *the major steps of a project*. It focuses on:

- Actors of each step
- Dependencies between steps
- *Sequencing*

Workflows assist in identifying *high-level* tasks and a sequence of *good* user tasks.

9. Object Oriented Design

Object Oriented Design (*OOD*) is a method of design encompassing the process of **object-oriented decomposition**. It views the system as *interactions of different objects*.

The goal of *OOD* is to find the important **object-based abstractions** of the system. We should ask questions:

- How can we **identify classes/objects**?
- How do objects **interact**?
- What **roles and responsibilities** do objects perform?.

9.1. Objects and classes

Objects are an **abstract model of a real-world entity** that has an independent **identity**. In design and implementation, it is a **component** that *contains a collection of data and related procedures*.

Objects can also **receive** and **response** to messages, and has behavior that may be dependent on its internal state, (the state can change by responding to messages).

Classes is a **description** of a **group of objects** with common data attributes, common operations, common relationships and semantics. Classes can be used to *create* objects called **instances**. A class should also capture **one and only one** key abstraction. If this is violated, it reduces *cohesion* (see Section 9.10).

Do not create god classes/objects. They have many responsibilities, with significant complexity and difficult to understand and maintain.

Classes can be described/represented in *UML*, using Mermaid Diagrams, i.e.:

```
classDiagram
    class Rectangle{
        - width: int
```

```

- length: int
+ calculate_area() int
+ calculate_parameter() int
}

```

where the plus signs mean public attributes and the negative signs mean private attributes.

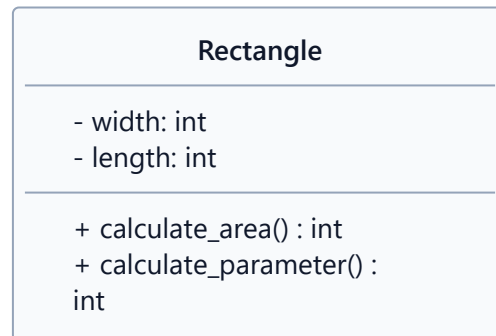


Figure 4: Sample UML diagram

9.2. Object-Oriented Design Process

The process then works with:

1. Find the **classes**
2. Determine the **responsibilities** of each class, and study the requirements specification;
3. Determine **how objects collaborate** to fulfill their responsibilities

Then

4. **Factor** common responsibilities to build class hierarchies
5. **Streamline collaborations** between objects.
 - e.g. is message traffic heavy in parts of the system?
6. Apply **design heuristics** to improve specific design aspects

The result in the design process is **not** a final product. Design decisions may be revisited, and it is rarely linear. Furthermore, design methods provide *guidelines*, not fixed rules. A good *sense of style* often helps produce *clean, elegant designs*, a.k.a designs that make sense from the engineering standpoint.

9.3. Finding classes

Identify:

- What are the **goals** of the system being designed, and the expected **inputs** and **outputs**?
- Look for **noun phrases**, separate into **obvious** classes, **uncertain** candidates, and **nonsense**.
 - Example noun phrases: **physical objects**, **conceptual entities**, etc.

CRC Cards can help with finding classes.

9.3.1. CRC Cards

CRC (*Class, Responsibility and Collaborations*) cards are used to capture and organize information about classes and responsibilities within a software system

Animal	
Responsibility	Collaboration
Model The Animal	Zoo

Table 2: Example CRC Card

CRC cards are used to **explore** possible designs, and **are not specification of designs**.

9.4. Responsibilities and Assigning Responsibilities

9.4.1. What are responsibilities?

Responsibilities are the **knowledge** an object maintains and provides, and the **actions** it can perform. It represents the **public services** an object may provide to client. Namely, it states what it does, **not** how it does.

To identify responsibilities, study the requirements specification, namely:

- Highlight **verbs**,

- Explore **alternative scenarios**
- Identify **actions** resulting from **different inputs**

Furthermore:

- Study classes, and use class names to infer roles and responsibilities.
- Use recorded purposes on cards (CRC cards?) as responsibilities

9.4.2. Assigning Responsibilities

Use Peirine's Laws:

- "Don't do anything you can push off to someone else."
- "Don't let anyone else play with you"

which means:

- **Evenly distribute** system intelligence (or *work*) *uniformly* among **top-level classes**:
 - avoid **centralization**, and keeps responsibilities close to objects.
- State responsibilities as **generally** as possible
- Keep **behavior** together with related information.
- Keep information about **one thing** in **one place**. If multiple objects need to access the same information, we need a **new object to manage, one obvious candidate, or collapse**
- **Break down complex responsibilities** and *share* responsibilities among related objects.
- **Maintaining information** *is* a responsibility

9.5. Relationships between classes

9.5.1. Example relationships

Additional responsibilities can be uncovered by examining relationships between classes.

Relationship Type	Definition
Is-Kind-Of	Classes sharing a common attribute often share a superclass $\Rightarrow$ common responsibilities
Is-Analogous-To	Similarities between classes can suggest undiscovered superclasses
Is-Part-Of	Distinguish responsibilities between <i>part</i> and <i>whole</i>

Table 3: Some kinds of relationships between classes

9.5.2. Association and Aggregation (and Composition)

- **Composition** is a “part-of” relationship where the **lifecycle of the contained object(s)** are **completely dependent on the container object**.
- **Aggregation** is a “has-a” relationship where **the contained object exists indepdently of the container**.
 - It is useful for building **modular and reusable** components.
- **Association** is a relationship between **two independent classes**, where an object uses or interacts with other object(s).
 - Associations can be either *unidirectional* or *bidirectional* and exists in forms as *one-to-one*, *one-to-many*, *many-to-one* and *many-to-many*

For good design, whenever there is a choice in an OOD between a **containment relationship** and an **association relationship**, pick the former, since it reduces amount of exposure of internals of a class¹

9.5.3. Collections

The elements of an association / aggregation generally needs some form of expandable container / collection in the corresponding enclosing object. Some notes of how to design collections:

- If you want to **avoid duplicates**, choose a **set-like collection**.

¹something something encapsulation

- If you want to **access elements by index**, choose a *indexable* collection (i.e. Array)
- If you want to **have a specific element order**, choose a collection that **allows ordering**.
- If you want some of the above, choose the collections that **supports all**.

9.6. Inheritance and Abstract Classes

9.6.1. Inheritance

Model Inheritance reflects “is-a” relations between abstractions in the model. **Software Inheritance** expresses relations between the software system.

Inheritance should be used **only** to model a **specialization hierarchy** (which classes are specializations of others?)

Theoretically, inheritance hierarchies can be *really* deep, but in practice, they are rarely deep, and should be no deeper than the short-term memory of a developer (5-7 layers).

9.6.2. Abstract Classes

Abstract Classes are classes that factor common behavior shared by other classes. It groups **related classes** with common responsibilities, and help introduces parent classes that represents the group.

9.6.3. How to design abstract classes

Good candidates for abstract classes are *categories*.

All abstract classes must be *base* classes (other classes derive from them). and all base classes should be abstract classes.

Concrete classes may be both instantiated and inherited from, while abstract classes can only be inherited from (i.e. **no** inheriting abstract classes from concrete classes). Sub-classes should **support all inherited**

responsibilities (and possibly more). They may refine or specialize a responsibility, but never hide them from the parent class.

9.7. Collaborations

In OOD, objects **collaborate** to perform the desired responsibilities.

For each responsibility, ask whether the class can fulfill the responsibility **by itself**, and **if not**, then:

- what does it **need**?
- From what *other* class can it obtain that?

For each class, ask:

- What does it **know**?
- What *other* class(es) need its **information** / **results**?

Some interaction considerations:

- **Minimize** the number of classes with which another class collaborates (i.e. don't have way too many collaborators for one class).
- **Minimize** the amount of collaboration between a class and its collaborators, (minimize the **number of messages sent**).
- **Do not** turn an operation into a class.
- Classes that **do not interact** with others should be discarded.

9.7.1. Roles and Protocols

Collaborations are often described using **protocols**. Each participant in a protocol plays a given **role**.

For example:

Drawing Element is a class of the graphics editor. It defines three *roles*:

1. *Client*: Uses other elements of the application
2. *Child*: Can be contained in a Group Element

3. *Subject*: state being observed by other elements.

Each of these roles implies a number of **related responsibilities**, and the usage of a particular **collaboration protocol**.

9.8. Design by Contract

Design by Contract is a design concept introduced by Bertrand Mayer in the late 1980s. It views the relationship between a **service provider** (an object) and **service consumer** (a *client* of the object) as a **formal agreement** (a *contract*) with rights and obligations.

It extends the definition of abstract data types (i.e. *classes*) with **pre-conditions**, **post-conditions**, **invariants**, and how they can be refined in classes.

In DbC:

- A *service consumer* must ensure that a service on a provider is only ever invoked **when the service's preconditions are met**.
- A *service provider* must ensure that **after successful completion of a service, the postconditions are met**.
- Invariants of the service provider must be held **after the successful execution** of any externally visible service(s).

-Subclasses can only **weaken service pre-conditions** or **strengthen service post-conditions**

The benefits of DbC are

- Explicit notion of contextualized correctness
- Enforces *separation of concerns*, which helps assign responsibilities to the correct class(es)
- Facilitates *testing* and *debugging*
- Enhances *documentation*

9.8.1. Abstract State v.s. Concrete State.

Every object contains **stateful information** (i.e. objects have **state**).

The externally visible properties of this state defines the **abstract state**.

The internal representations define the **concrete state**².

Clients should only rely on an objects' abstract state, and similarly object equality should only ever be defined over the abstract state.

9.8.2. Command-Query seperation principle.

All methods of a class should be divided into two non-overlapping categories:

Queries	Commands
Returns a result, without changing the observable state	Changes the (abstract) state without returning a result.

Table 4: Command-Query Seperation Principle

For example:

```
>>> a = [1, 2, 3, 4] # object declaration
>>> a.__len__() # query, returns a result, doesn't change the
object
4
>>> a
[1, 2, 3, 4]
>>> a.append(5) # command, doesn't return a result, changes
the object
>>> a
[1, 2, 3, 4, 5]
```

9.9. Decomposition

Decomposition (also called *divide-and-conquer*) is a process that helps to **manage the complexity** of a system.

²So it seems like *abstract state* = *public* and *concrete state* = *private*. What the hell happened to *protected*?

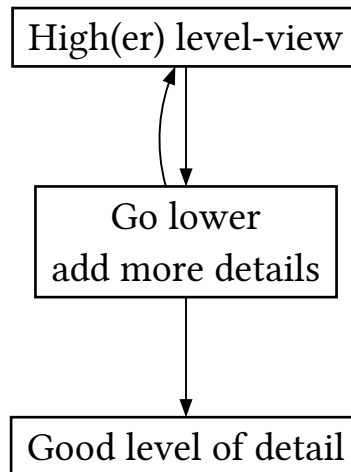


Figure 5: Decomposition Process

There are two types of decomposition: *functional* and *object-oriented* decomposition.

9.9.1. Functional v.s. Object-oriented Decomposition

Type	How to Decompose?	Issues
Functional	Decompose according to functions a system should perform	• Naive: modern systems perform more than one function • Maintainability: System functions evolve, design affects whole system • Interoperability: Interfacing with other system is difficult
Object-oriented	Decompose according to the objects a system is supposed to manipulate	Needs to identify objects

Table 5: Functional v.s. Object-oriented Decomposition

9.10. Coupling and Cohesion

Coupling and Cohesion are **measures** of **how well the parts of a system are tied together** to form the system itself.

Measure	Definition
Coupling	inter-dependency between system parts
Cohesion	intra-dependency of a system part

Table 6: Coupling and Cohesion

The goal of good OOD is to have **weak coupling** (system parts are independent and changing one shouldn't change others) and **strong cohesion** (code is tightly related, each module has one purpose)

Some types relevant to enterprise computing include:

Type of coupling	Problem description	Reduced by
Functional	One module relies on the internal workings of others	Well-defined API describing inputs/ outputs
Interface	Interfaces reflect the implementation details	Avoid detailed mandatory params, use more general methods
Data Structure	Data passed between modules reflects internal representations , or is over-constrained	Explicitly define formats, allow multiple representations
Temporal	Client blocked while waiting response, process over-constrained	Async messages, explicitly define behavior in interface
Address/URI	Addresses/Providers/URI change	Do not hard code references, virtualize services

Table 7: Some types of relevant coupling

10. Patterns

10.1. What are patterns?

Patterns are problems that **occurs over and over again**, and described the **core of the solution** to the problem. **Design patterns** are *standard solutions* to common software design problems.

The following are **not** design patterns:

- **Algorithms.** They solve *computation problems*, not *design problems*.
- **Software components.** Design patterns describe a *way* to solve a problem, and document pros and cons of implementations. Software components are *implementations* which may be implemented *using* design patterns
- **Frameworks.** Frameworks **implement** a “**generic**” **software architecture**, while a design pattern *documents* the solution to a specific problem. Frameworks may be *used and documented* with design patterns, and design patterns are **drawn from experience** with multiple applications solving related problems

10.2. What do patterns solve?

Software design patterns *document design experience* by:

- **Enabling widespread reuse** of software structures
- **Improving communication** within and across software teams
- **Capture knowledge** that experienced developers *already* understand implicitly.
- Arise from **practical experience**, and **faciliate** training of new developers.
- **Transcend** “**programming-language centric**” viewpoints.

10.3. Idioms, Frameworks, Libraries

An **idiom** is a **low-level pattern** specific to a programming language, and describes how to implement **particular aspects of components** or **the relationships between them** using features of the programming language.

Some example idioms include:

- **Delegation** is when an object **delegates** some of its work to another object. It is **more flexible, less structured** than interface; and is one of the *most basic* object-oriented idioms.
- **Interfaces**. Interfaces can be used to keep **clients** of a service **independent of classes that provide the service**.
 - In particular, if a **concrete class** is a service provider, then only instances of that class *or* its subclasses can be used in the future. By using interfaces, an instance of *any* classes that implement the interface can be used to provide the service.

10.4. Comparison of definitions

Term	Definition
Idiom	Idioms are common programming techniques and conventions , and are often <i>language-specific</i>
Pattern	Patterns document common solutions to design problems , and are <i>language-independent</i>
Libraries	Libraries are collections of functions, procedures or abstractions that can be used in many applications
Frameworks	Frameworks are open libraries that define the generic architecture of an application, and can be extended by adding or deriving new classes. Frameworks typically make use of common idioms and patterns

Table 8: Definition of Idioms, Patterns, Libraries and Frameworks

In traditional application architectures, **user code makes use of library functionality** in the form of procedures of classes. A framework *reverses* this relationship by providing both **generic functionality** and **application architecture**. Frameworks therefore calls user classes.

10.5. Design Patterns

A **design pattern** provides a **scheme** for **refining the subsystems or components** of a software system, or the relationships between them. It describes a **commonly-recurring structure** of **communicating components** that solve a **general-design problem** within a **particular context**

The format of a pattern consists of:

- **Pattern Name and Classification**, which should convey the essence of a pattern
- **The Problem Forces**, a.k.a when to apply the pattern:
 - **Intent**: Short Statement of rationale and intended use i.e.
 - **Motivation**: A problem scenario and example solution, i.e.
 - **Applicability**: in which solutions can the pattern be applied
- **The Solution Abstract** description of design elements:
 - **Structure**: Class and scenario diagrams
 - **Participants**: participating classes/objects and their responsibilities
 - **Collaborators**: how participants carry out responsibilities
- **The Consequences**: results and trade-offs of applying the pattern
 - **Implementation**: pitfalls, hints, techniques, language issues etc.
 - **Sample Code**: illustrative examples in various programming languages
 - **Known Uses**: examples of the pattern found in real systems
 - **Related Patterns**: competing and supporting patterns

10.5.1. Common Patterns

Some common patterns include:

- **Factory Method:** Introduce a Factory method in a base class/interface and delegate the decision with concrete classes to instantiate to subclasses
- **Strategy:** Define a **common interface** for the family of algorithms and have **specific implementations implement the interface**
- **Composite:** Define a **common interface** (or **abstract base classes**) that both **parts** and **composites** implement from.
- **Observer:** An **observer** object *publishes* state change events to its **subscribers**, who must implement a common interface for *receiving notification*.
 - Note that notification can be slow if there are *many observers for one observable*, or if *observers are observable*.
- **Null Object:** Implements the interface, but **does nothing**.
- **Singleton:** Introduce a **singleton** class that keeps track of a *static reference* to its only instance, and provide a static method to access this sole instance.
- **Command:** Encapsulate requests in Command objects that can be passed around and executed when ready.

10.6. Other kinds of patterns

Patterns are not limited to software designs, they can be found in many more problem domains:

- **Re-engineering** patterns
- **Usability** patterns
- **Documentation** patterns
- **Organizational** patterns

There are also *pattern systems*, since patterns **do not exist in isolation**, they always work together with other patterns. A plain catalogue of patterns does not reflect all relationships, they should be **intervoven**.

11. Software Architectures

A software architecture is a set of architectural (design) elements that have a particular form. Properties constrain the choice of architectural elements whereas rationale captures the motivation for the choice of elements and form.

— Dewayne Perry and Alexander Wolf, 1992

The architecture of a software system is (often) described as:

- The **structures of its high-level processing elements**
- The **Externally visible properties** of the processing elements
- The **Relationships** between them.

A **rationale** explains **why a decision was made** and **what the implications are in changing it**. It can be used to explain:

- **implications of systemwide design choices**
- **effects on the architecture** in the context of add/removing requirements

11.1. Subsystems, Modules, Components

A **sub-system** is a **system** (in its own right) whose operation is **independent** of the services provided by other subsystems.

A **module** is a system component that **provides services** to other components, but would **not** normally be considered as a separate system.

A **component** is an independently deliverable unit of software that *encapsulates its design and implementation*, and offers interfaces to the

outside; by which it may be composed with other components to form a larger whole.

11.2. Why use architectures?

Architectures are the **technical interface** between the customer and the contractor building a system. A bad architecture **can not be rescued by good constructions**. There are **specialized types**, and **styles** of software architecture.

Furthermore:

- Architectural styles document **existing, well-proven** design experience
- Architectures **identify and classify** abstractions that are a **higher** level of abstraction compared to simple programming language constructs
- Architectural styles provide a **common vocabulary** and **understanding** for design principles.
- Reference Architectures support the construction of software with well-defined properties.
- Guidelines help to choose suitable architectures given the problem domain or required quality attributes.

⇒ Architectures help to manage software complexity.

11.3. Architectural Styles

An **Architectural Style** defines a **family of software systems** in terms of their **structural organization**. It expresses **components and their relationships** with the constraints of the application, and the associated *composition and design rules* for the construction.

Some popular architectural styles are:

- **Data-flow architectures:** Batch Sequential, Pipes-and-Filter

- **Call-and-return architectures:** Client-Server, Layered, Object Oriented, etc
- **Data-centered architectures:** Repository, Blackbaord
- **Virtual machine architectures:** Interpreter, rule-based systems
- **Independent Component Architectures:** P2P, Event-Driven

11.3.1. Layered Architectures

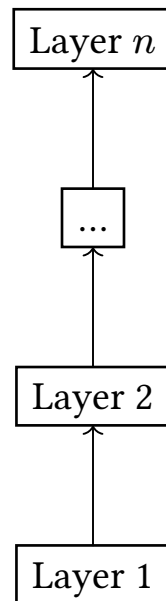


Figure 6: Visualization of Layered Architecture

A **layered architecture** organizes a system/module/component into a **set of layers** each of which **provide a set of services to the layer above** and **uses services from the layer below**.

Elements in each layer can only see other elements in the same layer, or the layer below. Callbacks may be used to communicate to higher layers.

11.3.2. Dataflow Architecture

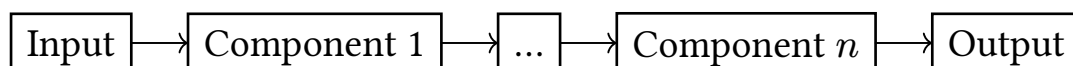


Figure 7: Visualization of Dataflow Architecture

In a data-flow architecture, each components perform **functional transformations** on its inputs to **create outputs**.

Its main elements include:

- Data **sources** and data **sinks**
- **Filters** and **Transformers**

The data is organized so that each processing element (a *filter*) is discrete and only carry *one* type of data information. The data flows from one processing element to another for processing. In general, there is a single input and output.

11.3.3. Repository Architectures

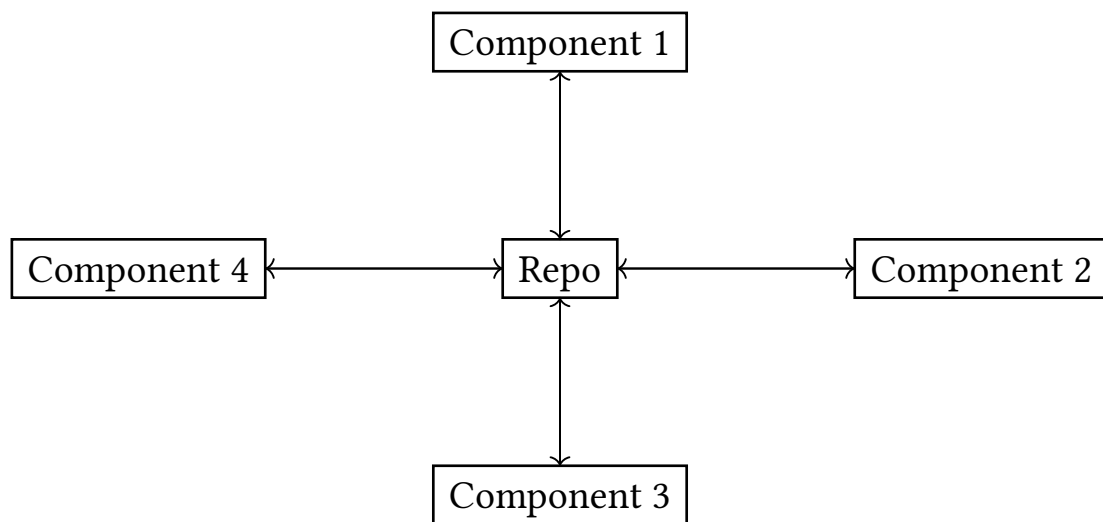


Figure 8: Visualization of Repository Architecture

In a **repository architecture**, all data is managed in a **central repository** that is **accessible to all system components**. Components do not interact directly, only through the repository.

However, this means that components must agree on a repository data model, there is no scope, and it's difficult to distribute evenly. Examples are DB-based information systems, and document repositories.

11.3.4. Client-server Architectures

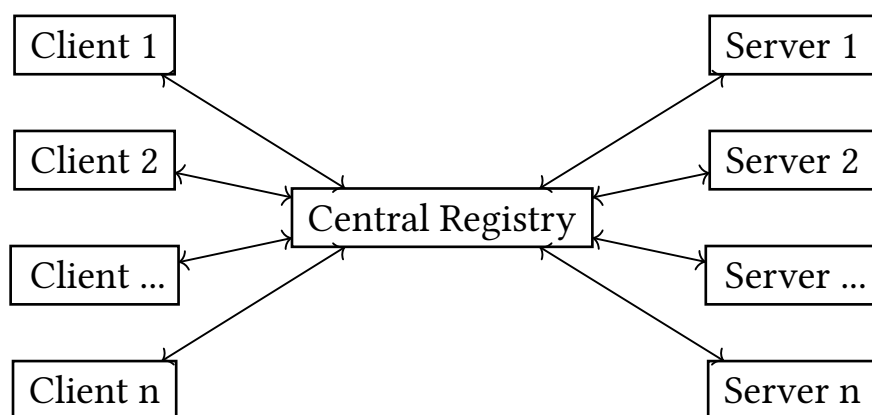


Figure 9: Visualization of client-server architecture

In a **client-server architecture**, application logics and services are **distributed to a number of client and server subsystems**, each (potentially) running on a different machine and communication through the network.

Some advantages are straightforward distribution data, effective use of networked systems, and easy to add new servers. Some disadvantages are lack of shared data model, redundant management, and require a *central registry* of available names and services.

11.3.5. Event-driven architectures

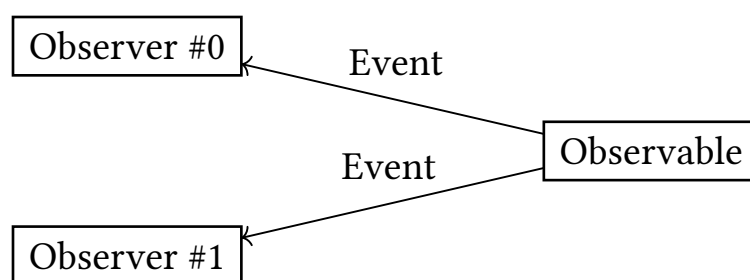


Figure 10: Visualization of client-server architecture

In an **event-driven architecture**, components perform services **in reaction to external events** generated by other components. It has some models:

- **Interrupt-driven:** **real time interrupts** are detected by an **interrupt handler** and passed to some other component for processing.

- **Broadcast:** an event is broadcasted to all subsystems, and any system that can handle the event will handle it.

11.3.6. Peer-to-peer architectures

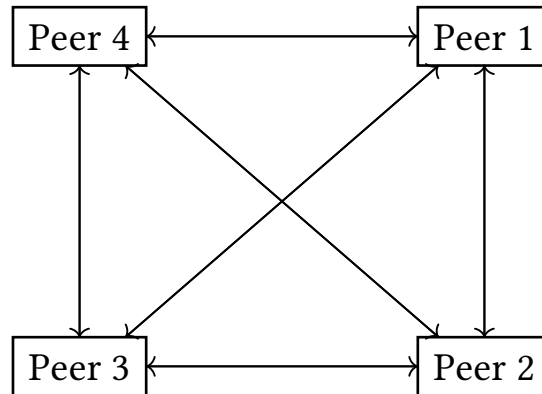


Figure 11: Visualization of Repository Architecture

A **peer-to-peer** architecture is a type of **decentralized** and often **distributed** structure in which individual **nodes** are both as suppliers and consumers of resources. In a P2P network, tasks are often shared amongst multiple **inter-connected peers** who each make a portion of their resources available to other peers, **without centralized coordination**.

11.4. Architectural Tactics

An **architectural tactic** is a **transformation on an architecture** or a **change to the input** to a system that results in the **improvement** of specific quality attributes.

Examples:

- **Information Hiding** is a transformation on an architecture that improves **modifiability**
- **Redundancy** is a transformation on an architecture that improves **availability, reliability, and/or performance**.
- **Reducing the arrival rate of requests** is a change to the input of a system that improves **latency/response time**.

Examples of various tactics:

- **Scalability:** Scalability consists of **vertical scaling** (increase **power** of existing hardware) and **horizontal scaling** (increase **quantity** of hardware)
- **Availability**
- **Modifiability**
- **Performance**
- **Security**

Patterns implement several tactics, and any implementation of patterns also makes choices about tactics.

11.5. Other

11.5.1. Heterogenous Styles

Large Software Systems **rarely confirm** to one single architectural model or style. They incorporate different styles at different levels of abstraction.

- **Locationally Heterogenous:** Different areas of a systems have different structures
- **Hirerarchitcally Heterogenous:** A component of one style, when decomposed, is structured like a different stlye.
- **Simultaneously Heterogenous:** A system can be described with various architectural styles.

11.5.2. Reference Models and Architectures

A **reference model** is a division of functionality together with data flow between the resulting elements.

A **reference architecture** is a reference model mapped into software components and data flows between those components.

12. Web Services

12.1. Definitions

In order to introduce Web Services, we need some definitions:

- **The Web**, which consists of:
 - **Documents**, that are hosted by **servers** and read/used by **clients**

The web is **distributed**, **searchable**, **addressable**, **extensible** and **unbounded**

Enterprise (software) systems are systems that are *distributed*, featuring **business process** that are **cross-organization**, **cross-geography**, **large-scale** and **addressable**.

A **software service** is beyond a component, that is, it is **independently deployable**, **self-contained**, **self-describing**, **discoverable**, **composable**. Software services *do not have to be* on the web.

12.2. Concepts

Some other concepts include:

- **Functional Description** details the operational characteristics that define the overall behavior of the service
- **Non-functional Description** targets service quality attributes, i.e. *cost*, *performance metrics* (response time / accuracy), *security*, etc
- **Stateless services** are services that can be invoked repeatedly **without having to maintain context or state**.
- **Stateful services** are services that **require** their context to be **preserved** from one invocation to another.

Granularity is the quality of **including a lot of small details**

— Cambridge Dictionary

- **Service granularity:** Simple services are discrete (*atomic*) in nature, while complex services are *coarse-grained*, as these involve interactions with other services and possibly end-users in single or multiple sessions.
 - Coarse-grained communication implies **larger and richer** data structures (i.e. XML, JSON)
- **Synchronous communication** is **synchronized** between two communicating systems which **both** must be **up and running**.
 - This means that execution at the client's side is **interrupted** to execute the call
- **Asynchronous communication** where the caller employs a **send-and-forget** approach that allows it to continue after it send the message.
 - This means that execution at the client's side is **continued** after it sends the request.
 - The sending application **assumes** that the **message will arrive safely** at its **destination** at some point in time.

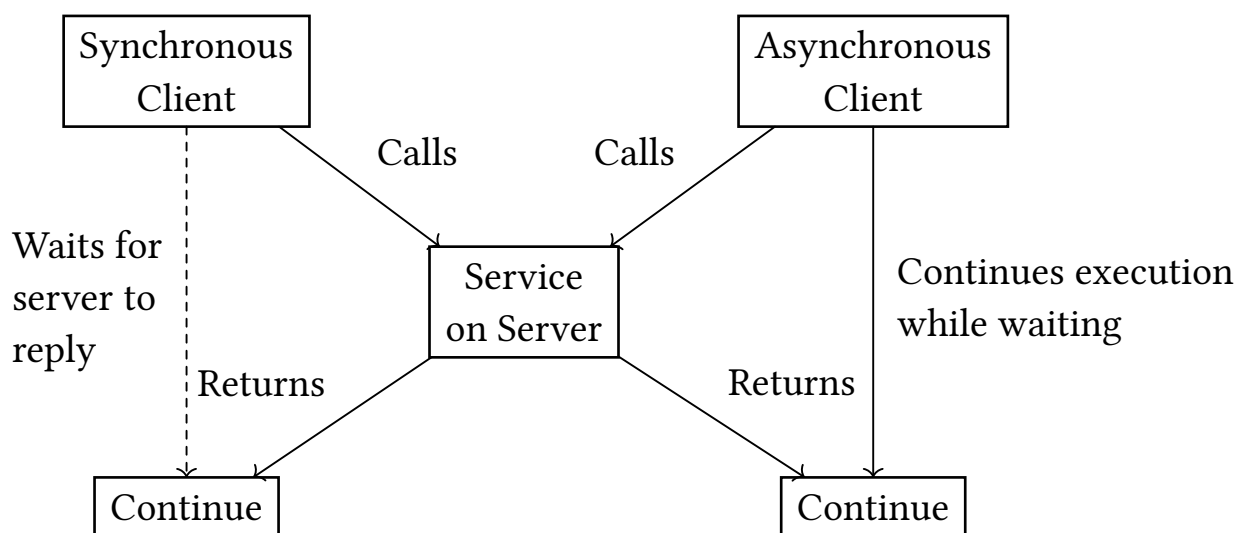


Figure 12: Sync vs Async communication

- The **Service interface** defines service **functionality** that is **visible to the external world** and **provides the means to access** this functionality.

- The services describes its interface, i.e. the operations available, parameters, data types, etc.
- The **Service Implementation** realizes a specific **service interface** whose implementation details are **hidden** from its users.

12.3. Service-Oriented Architecture (SOA)

SOA is fundamentally based on **client-server** style, and based on interaction between **decoupled services** and **end-users application**, that are associated with **messages** and governed by **policies**.

These services are **distributed in a network**, via **published and discoverable** interfaces.

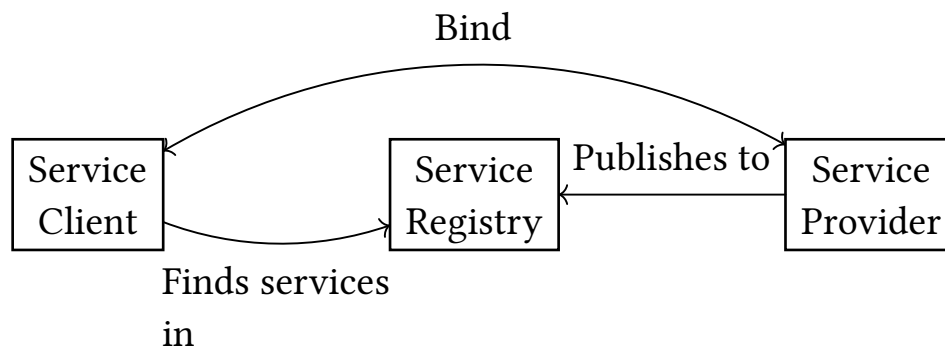


Figure 13: SOA Visualization

SOA is based on **client-server**, which has the following sub-styles:

- **Remote Procedure Calls (RPC)**: It is an **object-oriented approach** to distributed computing, and uses a **synchronous** communication. RPC-style programming leads to **tight coupling** of interfaces and applications.
 - In an RPC environment each application needs to **know the intimate detail** of the **interface** of every other application.
- **Message-Oriented Middleware (MOM)**: It is an architecture that involves **passing data between applications** using a **common communication channel** that carries **self-contained messages**. Messages are sent and received **asynchronously**.

- The **messaging system** is responsible for **managing the connection points between clients** and for **managing multiple channels of communication** between the connection ports.
- **Resource-oriented Architecture:** The **resources** are exposed on a network, uniquely identified. The **application** is a **series of linked resources updates** between **clients** and **servers**.
- **Event Driven Architectures (EDA):** The architecture depends on event **emitters** and **consumers**, which promote **further decoupling** of business processes. One event can be consumed by various processes, and emitters don't care who consumes the event.

12.4. Web application architectural styles and middleware

There are two standard approaches to web services, and different “protocol patterns” suit various architectural styles:

12.4.1. Web Service Standard: WS*, SOAP and WSDL

Web Service Standards (WS*) is based on the evolution of the **enterprise computing middleware** approach: i.e. MoM. **Standards** are needed, so the (enterprise) systems can interoperate. It defines a standard **lightweight XML-based** messaging format called **SOAP** (Simple Object Access Protocol), that can use multiple **application protocols** such as HTTP, HTTPS, etc, for transport.

SOAP allows applications to pass **messages and data** back and forth between disparate systems in a **distributed environment**, enabling **remote method invocation**. Messages are seen as **envelopes** where the application **encloses the data** to be sent. A SOAP message consists of an <Envelope> element containing an optional <Header>, which contains **blocks of information** relevant to how the message is to be processed, and a mandatory <Body> element which contains the main end-to-end conveyed information. The contents of these elements are not a part of the SOAP standard.

It is **lightweight**, since it only possesses two fundamental properties:

- Send and receive **HTTP** (or other protocols) packets
- Process **XML** messages.

SOAP overcomes problems of **conventional distributed object models**:

- It **does not require** uniform object model accross applications
- It features **loose-coupling** and **strong encapsulation** for applications.
- It is designed to work with WSDL
- It uses **HTTP**, which work over firewalls or proxy servers

Many big vendors develop middleware products to support the standards-based approach. An **application protocol** (SOAP) views/uses **HTTP** as a **transport protocol** to carry its messages.

It also has **calls/messages** defined and exchanged by the machine using **XML**.

There are four common types of operations that represent possible combinations of input/output messages:

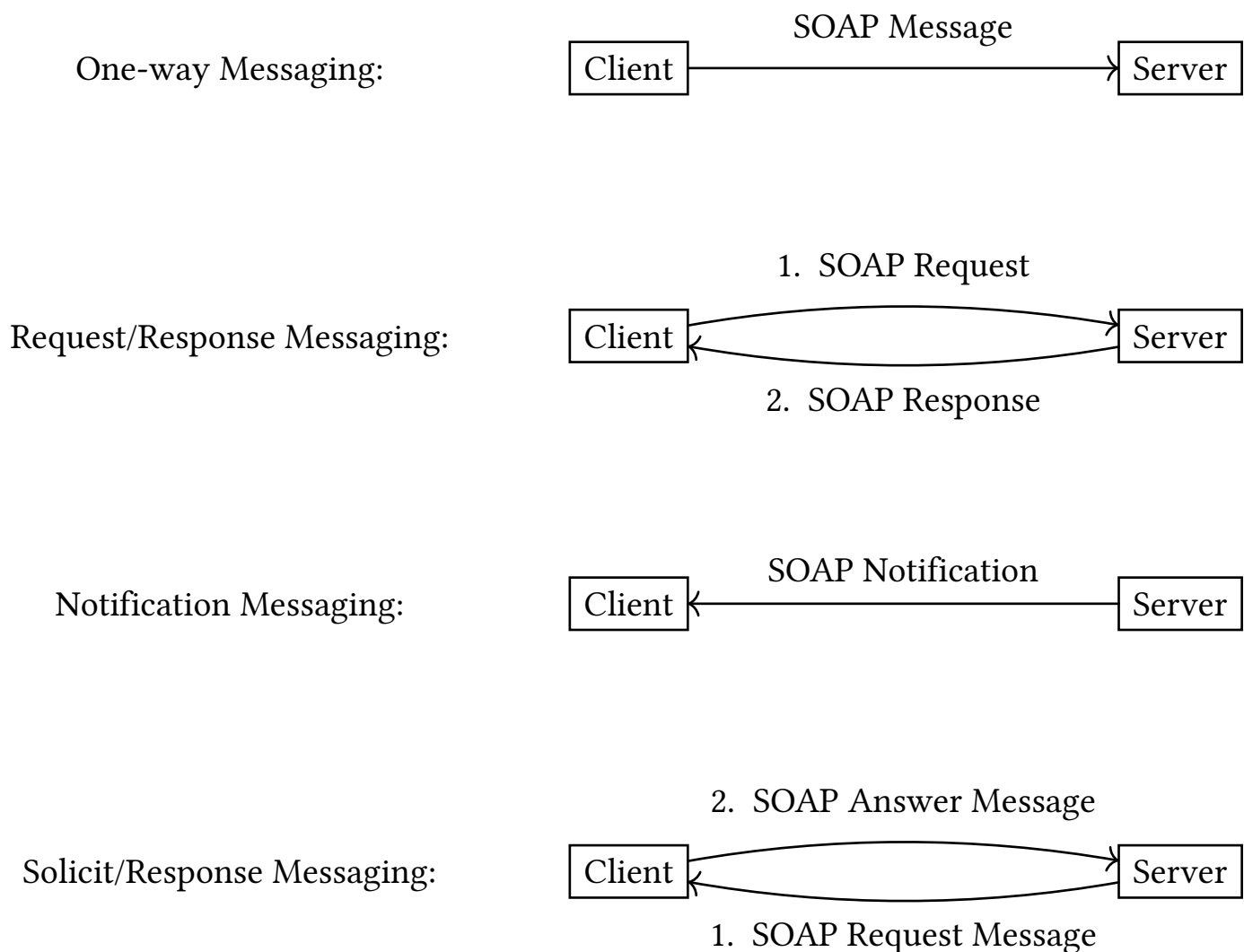


Figure 14: Message Exchange Patterns in WS*

Note that in **request/response messaging**, the client **request** happens **before** the server **response**; and in **solicit/response messaging**, the server **request** happens **before** the client **answer**.

In order to work with SOAP, we use **WSDL** (Web Service Description Language). WSDL is a **XML-based service representation language** used to describe the **details** of the **complete interfaces** exposed by **Web Services**, and is therefore **the means to access a Web Service**. WSDL is **platform-independent** and meant to be created and read **by machines**.

WSDL documents can be separated into distinct **sections**:

- The **service interface definition** describes the **general web service interface structure**, which contains all the **operations** supported by the service, its **parameters** and **abstract data types**.

- The **service implementation part** binds the **abstract interface** to a **concrete network address**, to a **specific protocol** and to **concrete data structures**.

This enables each part to be defined **separately** and **independently**, and **reused** by other parts. The combination of these two parts contains **sufficient information** to describe to the **service requester** how to **invoke** and **interact** with the service at a **provider's site**.

To implement the WS* standard, one needs to:

1. **Design and implement** an **object-oriented system**
2. **Generate** the **WSDL interface**
3. **Deploy** the service
4. **Publish** the WSDL interface (with APIs)

12.4.2. REST: (Representational State Transfer)

REST (Representational State Transfer) is a web architecture based on the **client-server architecture** that has **Purist** and **Hybrid** approaches. In **REST**, the **application state** is driven by the **client** updating resources across the **resource network** using **links** provided by the servers. It is a **style** or **design guideline**, not a standard.

Resources is something returned by an SQL Query, text file, etc.

In the **Purist** approach, REST features:

- **HTTP** as an **application protocol**
- **Resource-oriented** architecture, as everything is referenced by **URIs**.
- HTTP “verbs” (*requests*) have specific meaning:
 - **GET**: Retrieve information on a resource
 - **POST**: Create a new resource
 - **PUT**: Modify an existing resource
 - **DELETE**: Delete an existing resource

REST can be implemented by:

1. **Design and implement** an object-oriented software system

2. **Deploy the service** (with API)
3. **Publish API interface**

REST has 6 **constraints**:

- **Uniform interface**: there should be an uniform way to interact with a given server irrespective of device or type of application
- **Stateless**: The state **necessary** to handle the request is **contained** within the request itself.
- **Cacheable**: Every response should include whether the response is **cacheable** or not.
- **Client-server**: REST applications should have a **client-server architecture**, where **clients** request resources and **servers** holding them.
- **Layered system**: REST application architectures should be composed of **multiple layers**, with each layer **not knowing anything** about layers **other than its immediates**.

12.4.3. Microservices

Microservices are **modular, self-contained, deployable, scalable**, etc.

13. Documentation

13.1. Purpose of Documentation

The purpose of documentation includes:

- **Communication**: which communicate **ideas** about **possible software solutions** to stakeholders
 - Who is the audience? What do they need? What means do they understand?
- **Specification: Blueprint** for implementation
 - Provide a detailed **design specification** for implementers, need to avoid **ambiguity**.

The stakeholders include, (but not limited to): customers, end-users, developers, maintainers, business analyst (BA), system administrator (sysadmin), etc. All have **different interests and needs** for documentation

13.2. View:

A view is a representation of a coherent set of architectural elements, as written by and read by stakeholders. It consists of a representation of a set of [architectural] elements and the relationships amongst them

— Len Bass et al

.

Views focus on **specific aspects** of a software system by using appropriate **abstractions**.

13.2.1. Categories of Views

Category	Focuses on	Question
Component and connector	Runtime computational entities (i.e. components), and their means of communication (connectors)	What are the main executing components and how do they interact ?
Allocation	Computational entities and their environment	What processing node does each component execute on ?
Module	Code units and their responsibilities	What packaging mechanisms are used to specify/implement each architectural elements ? How are these packages inter-related ?

Table 9: Categories of views

13.2.2. Kruchten's 4+1 View

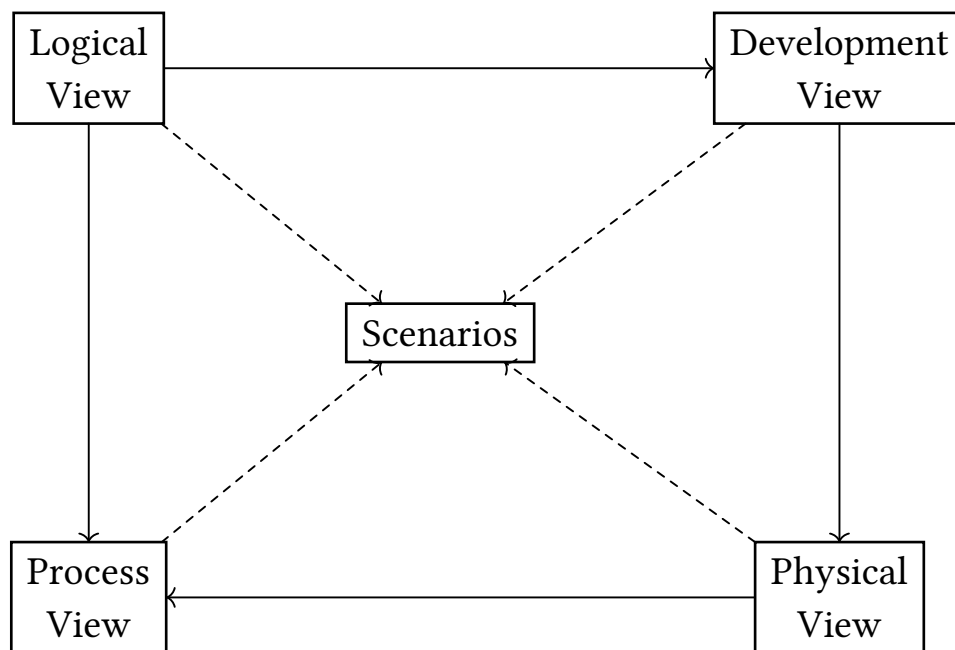


Figure 15: Visualization of Kruchten's 4 + 1 view

Kruchten's 4 + 1 view includes:

- **Logical View** (Module View): concerned with the **functionality** that systems provide to **end-users**.
- **Process View** (Component and Connector View): focuses on **system processes** (i.e. **components**) and how they **communicate**; address **concurrency, distribution, performance**, etc
- **Development View** (Allocation View): **illustrates** a system from a **developer's perspective**, concerned with **software management**.
- **Physical View** (Allocation View): concerned with **mapping of components** onto **processing** and **communication** nodes

Scenarios are **illustrations of architectural solutions** using **selected use cases**. It **describes sequences of interactions** between **system processes**, used to **illustrate** and **verify** the architectural design, and serve as a starting point for **tests** and **architectural prototypes**.

13.3. IEEE Standard 1471

IEEE 1471 provides definitions and a **meta-model** for the **description** of software architectures. It assumes that an architecture exists to address specific **stakeholder concerns**, and that **architectural descriptions** are **multi-view** (no single view captures all stakeholder concerns).

It separates the notion of **view** from **viewpoint**, where a **viewpoint** identifies the **set of concerns** and the **representations/modeling techniques**, used to describe an architecture to address those concerns. It provides for capturing **rationale** and inconsistencies/unresolved issues between the views within a single **architecture description**.

Viewpoints are specified by **name** (i.e. “Capability”), the **stakeholders addressed** (i.e. “clients, producers, developers”), the **stakeholder concerns** to be addressed (i.e. “how is functionality packaged?”), the **viewpoint language / modeling, etc** (i.e. components via UML diagrams), and the **source**.

Views are **well-formed**: each view corresponds to **exactly one viewpoint**, a **viewpoint** is a **pattern** for constructing **views**, and define the **rules** on **views**.

IEEE 1471 is “agnostic” on where viewpoints come from, it defines templates how viewpoints are declared; and each concern of a system is addressed by one or many architectural views.

13.4. Notations

Graphical Notations are a popular approach to document **software architectures** using **graphical notations**. They are generally easy to understand and use, but if not used systematically, may lead to ambiguity. They do not lend themselves for further analysis. Their main purpose is **communication, not specification**

The *de facto* standard for graphical notation is **UML**, which **reduces risks** by **documenting assumptions** (domain models, requirements,

etc), represents **industry standard** (which means more tool support, more understanding), is “reasonably” **well-defined**, and **open** (stereotypes, tags and constraints to extend basic constructs).

There are also **architectural description languages**, which can be characterized as very-high level “programming” languages. They exhibit many properties of a programming language, but **may not be computationally complete**. They feature basic building blocks for **high-level architectural elements** such as **components, connectors, configuration**; and *subsume* some **formal semantic model**. Their main purpose is **specification, not communication**

13.5. Design Evaluation

Evaluating design helps:

- Early detection of **design flaws**, which can **improve quality of designs** and presents opportunities to **fix flaws before code is written**.
- Early verification of **architectural drivers**, with a focus on **quality attributes**.
- Enhance **motivation** for improved design documentation
- Validate **requirements**

⇒ Validate both **quality** and **risks**

The benefits are:

- Improved **articulation** and **verification** of **critical architectural drivers** and **quality attributes**; therefore improving **software quality**
- Forces **clear explanation** and **documentation** of **critical design issues**, in particular at an architectural level; therefore improving **architectural rationale**
- Enhances **communication** between **stakeholders**.

and costs are:

- Time
- Money
- ...Frustration

13.5.1. Architecture Tradeoff Analysis Method

We can use the **Architecture Tradeoff Analysis Method (ATAM)**, which is a “heavy weight method” to systematically analyze software architectures. It is a comprehensive process with **5 phases / 9 stages**, involves **most stakeholders**, take up **several days**, and delivers **comprehensive risk analysis** of a given architectural design.

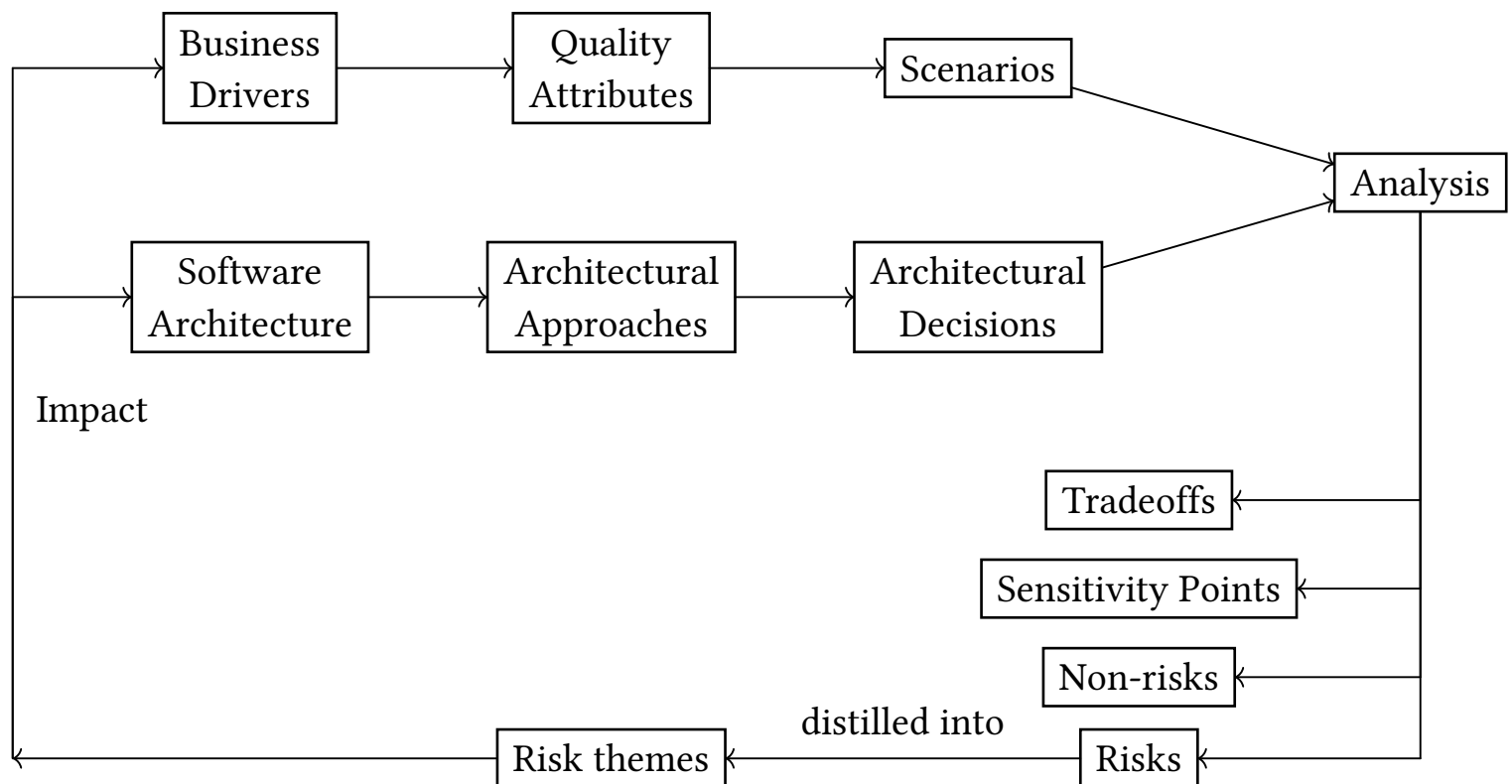


Figure 16: Visualization of ATAM

The output of ATAM is a **concise presentation** of the architecture, the defined and evaluated **business goals**; **quality requirements** via a **collection of scenarios**; mapping of **architectural decisions** to **quality requirements**, and a set of **risks, tradeoffs, and sensitivity points**.

The ebenfits of ATAM include:

- Get **stakeholders** together
- Enforces critical **quality goals**
- Prioritization of **conflicting goals**
- Forces clear **architecture presentation/rationale**
- Improves quality of **architectural documentation**
- **Risk identification** early in the development lifecycle

However, ATAM is “too heavy”, therefore, we’ll identify “the essentials”:

- **Architectural drivers**
- Prioritized **quality attributes**, using a **quality utility tree**
- **Scenarios**
- **Risk Assessment.**

13.5.2. Quality Utility Tree

A **quality utility tree** is built to **identify and prioritize** the most important **quality requirements**. It works by

1. Select the **most important quality attributes** to the **high-level nodes** (performance, modifiability, security, etc)
2. Add specific **quality requirements** at the next level
3. **Scenarios** are the **leaves** of the utility tree
4. Assess leaves based on **importance** and **degree of difficulty**

It results in a **characterization** and a **prioritization** of specific quality attribute requirements.